

5. Header Linked List

–Insert At Position

Program:

```
void insertAtPosition(int data, int pos)
{
    if (pos < 1 || pos > head->data + 1)
    {
        cout << "Invalid position! Current list size: " <<
head->data << endl;
        return;
    }

    Node *newNode = (Node *)malloc(sizeof(Node));
    if (newNode == nullptr) {
        cout << "Error: Memory allocation failed." << endl;
        return;
    }
    newNode->data = data;
    Node *temp = head;
    for (int i = 1; i < pos; i++)
    {
        temp = temp->next;
    }

    // Insert the node
    newNode->next = temp->next;
    temp->next = newNode;
    head->data++;

    cout << data << " inserted at position " << pos << "."
<< endl;
}

...

case 3:
    cout << "Enter data to insert: ";
    cin >> data;
```

```
cout << "Enter position: ";  
cin >> pos;  
insertAtPosition(data, pos);  
break;
```

Program Analysis

Now, if list has 4 nodes, Header Node's Data is also 4.

Now we can insert header node's data at position: 1, 2, 3, 4, 5. Greater than that of five (5) will result: Invalid Position.

Therefore, when `INVALID POSITION` : When $POS < 1$ or $Pos > Header\ Node's\ Data + 1$.

```
Node *newNode = (Node *)malloc(sizeof(Node));
```

→ The size of newNode will be equal to the size of a Node structure and newNode is declared as a pointer to Node structure.

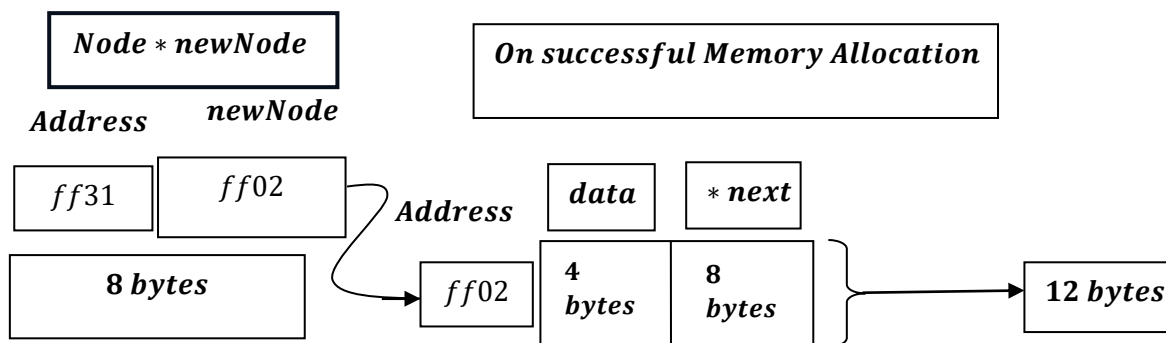
→ The size of a pointer (newNode) is typically 8 bytes on most modern 64 – bit systems, regardless of the size of the structure it points to.

→ *The size of the Node structure itself is determined by the sum of the sizes of its members:*

→ *data(an integer) is typically 4 bytes.*

→ *next(a pointer to another Node) is typically 8 bytes.*

→ *Therefore ,the size of the Node structure is typically 12 bytes (4 bytes for data + 8 bytes for next) .*



```
if (newNode == nullptr)
{
    cout << "Error: Memory allocation failed." << endl;
    return;
}
```

Now newNode = ff04, hence → Memory Allocation is Successful !

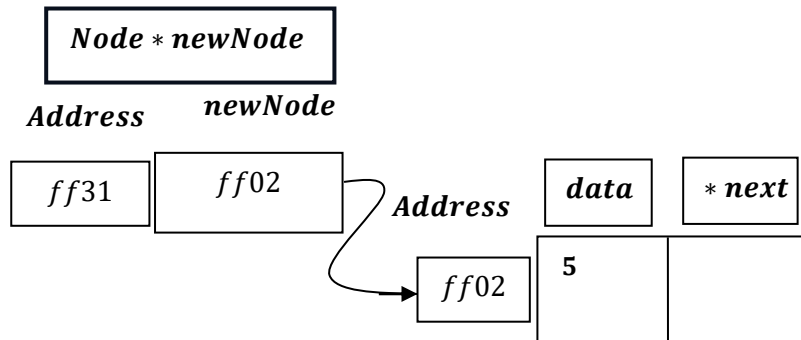
if newNode = NULL , then :

Message → "Error: Memory Allocation for Header Failed."

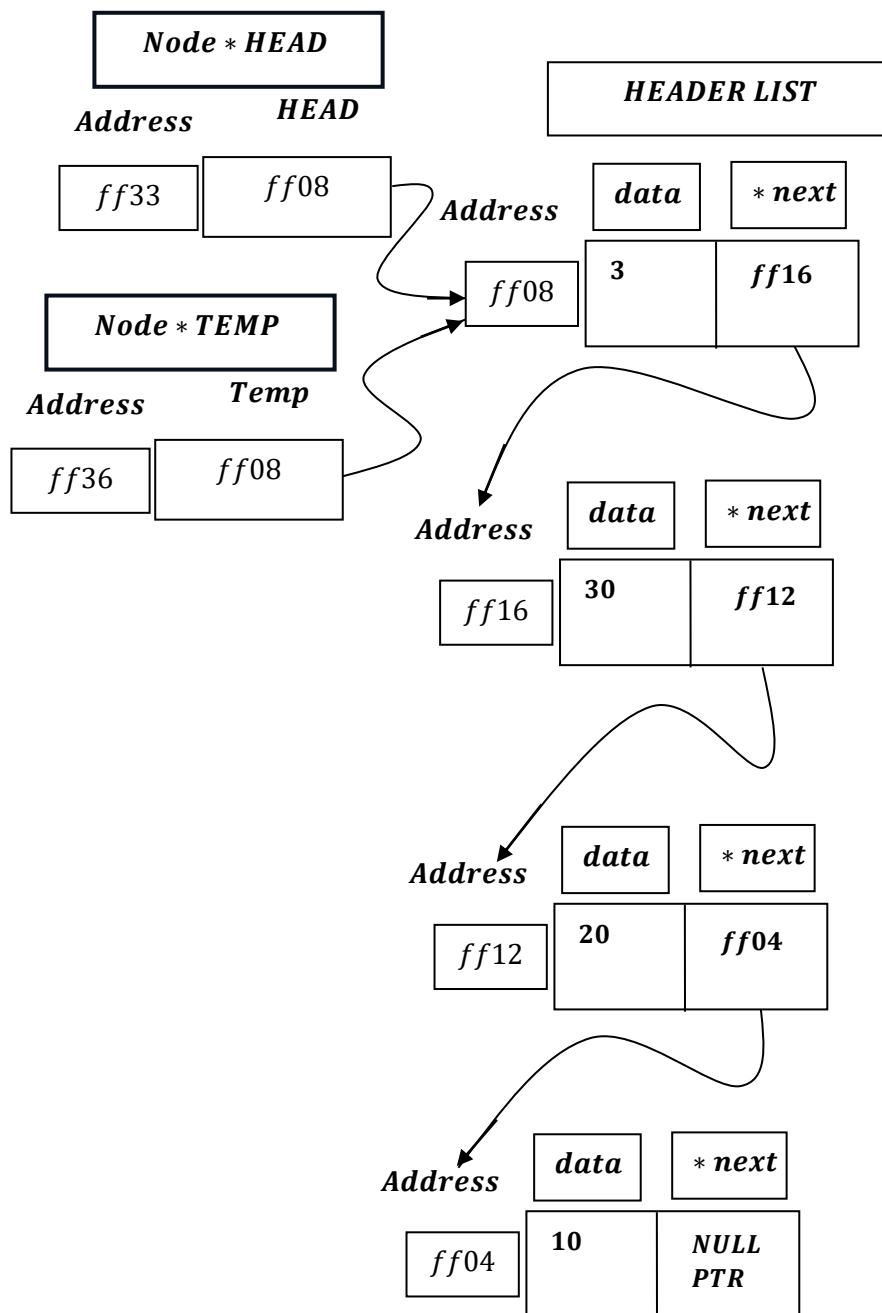
And then reurn will return void and exit from the module (function).

```
newNode->data = data;  
Node *temp = head;
```

Let data = 5.



And , Node * temp = head;



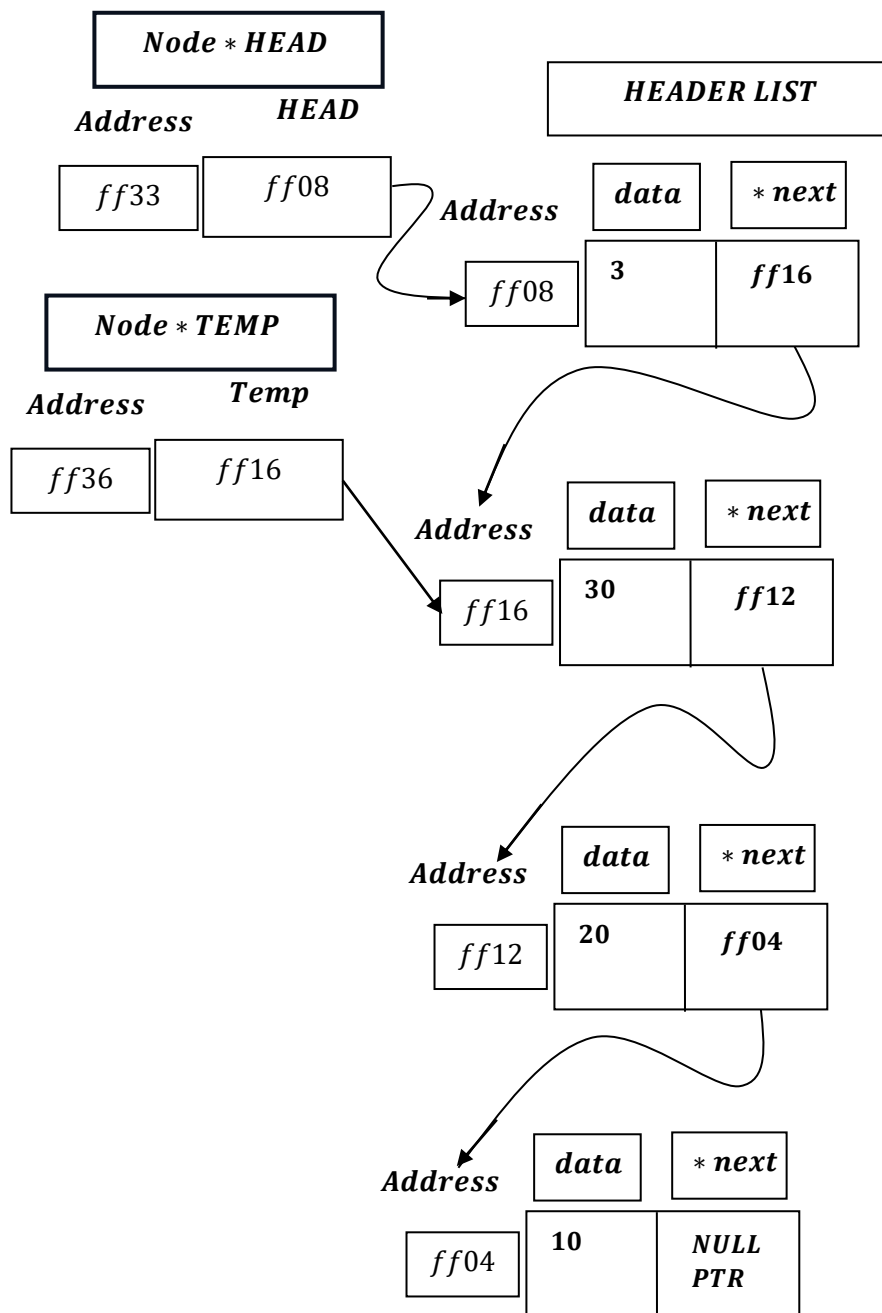
```
for (int i = 1; i < pos; i++)  
{  
    temp = temp->next;  
}
```

say, Position `pos`, pos: 3.

Hence,

$i = 1$ and $i < pos = 3$:

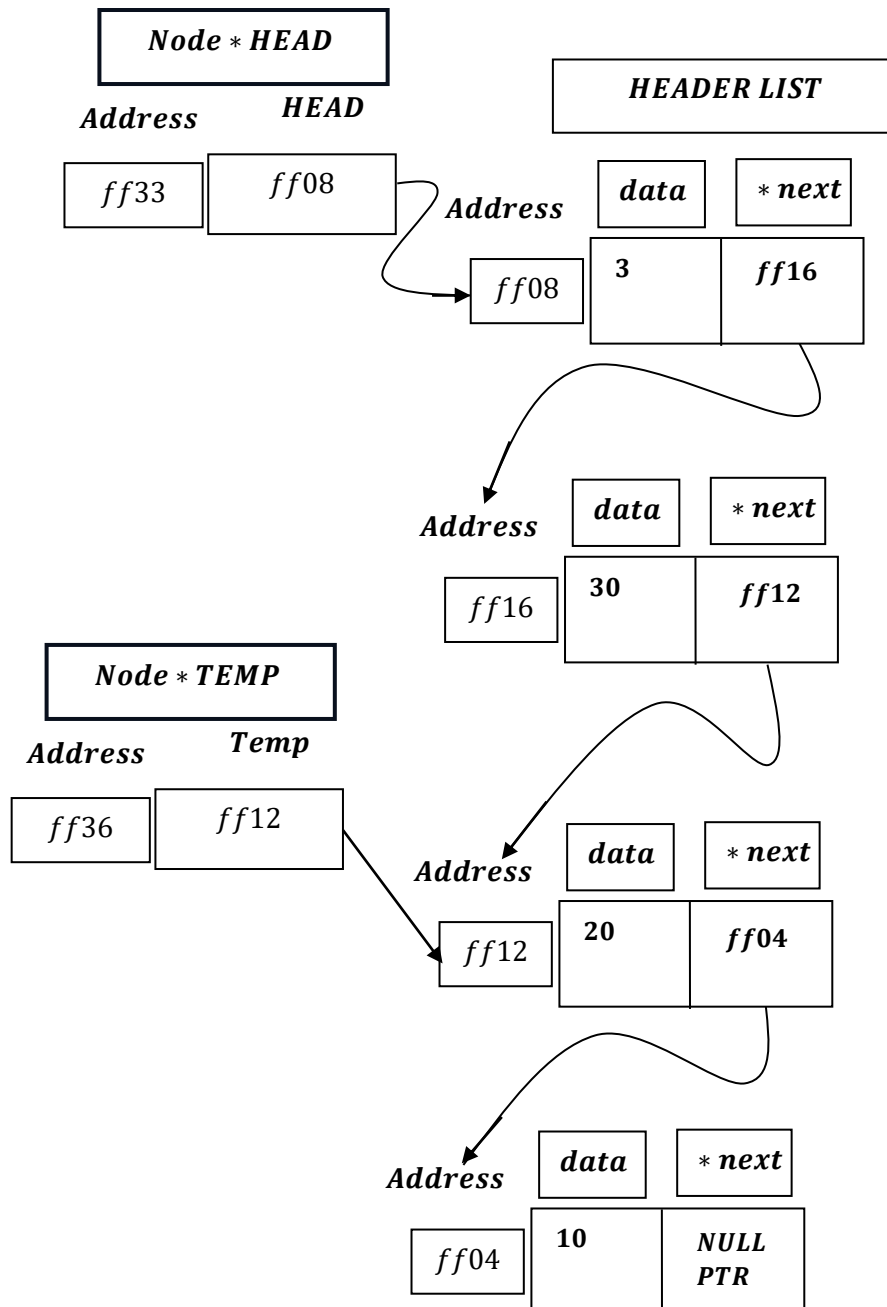
$temp = temp \rightarrow next = ff16$



$$i = i + 1 = 1 + 1 = 2.$$

$i = 2$ and $i < pos = 3$:

$temp = temp \rightarrow next = ff12$

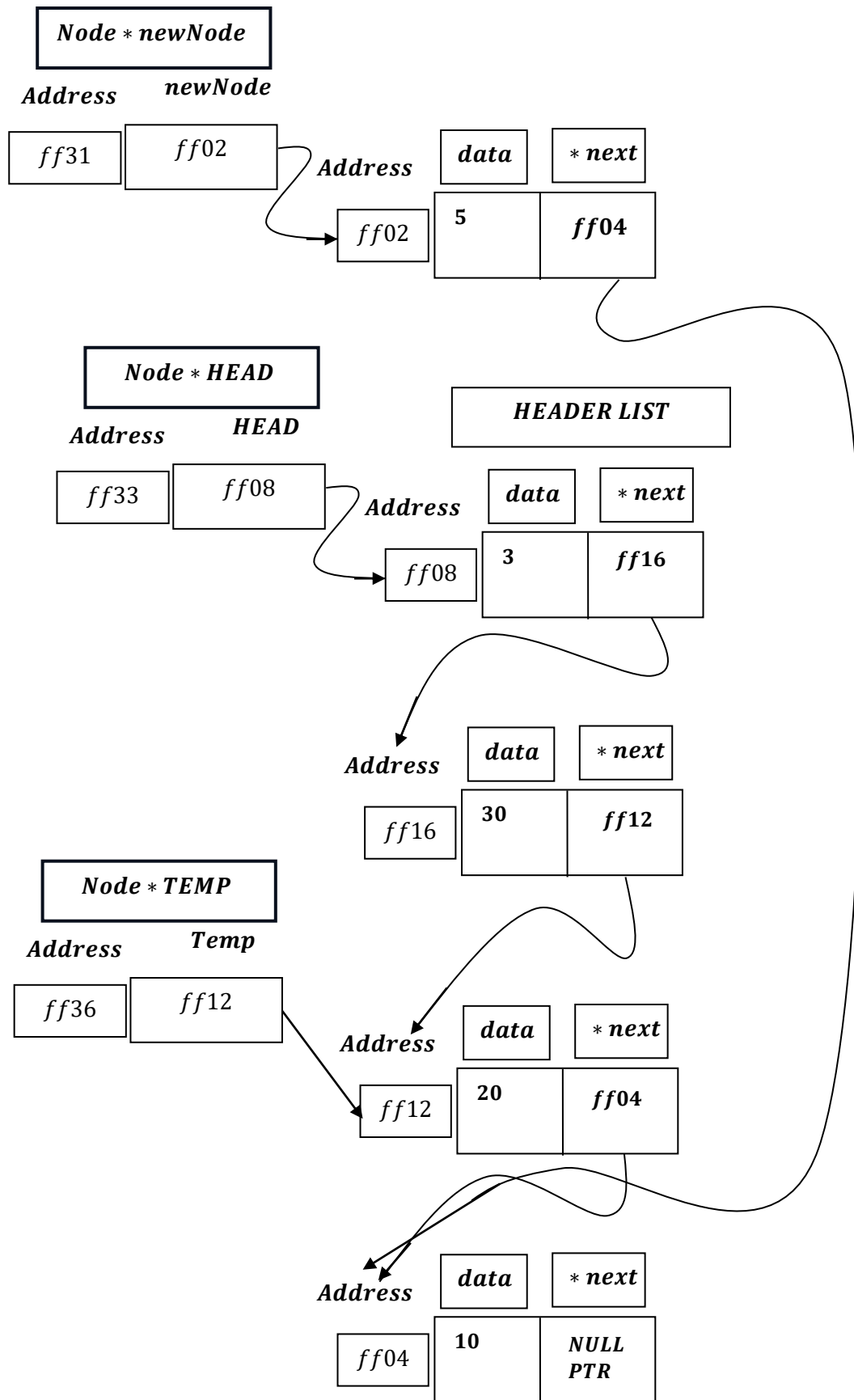


$i = i + 1 = 2 + 1 = 3.$

$i = 3$ and $i = 3 < pos = 3$ is false.

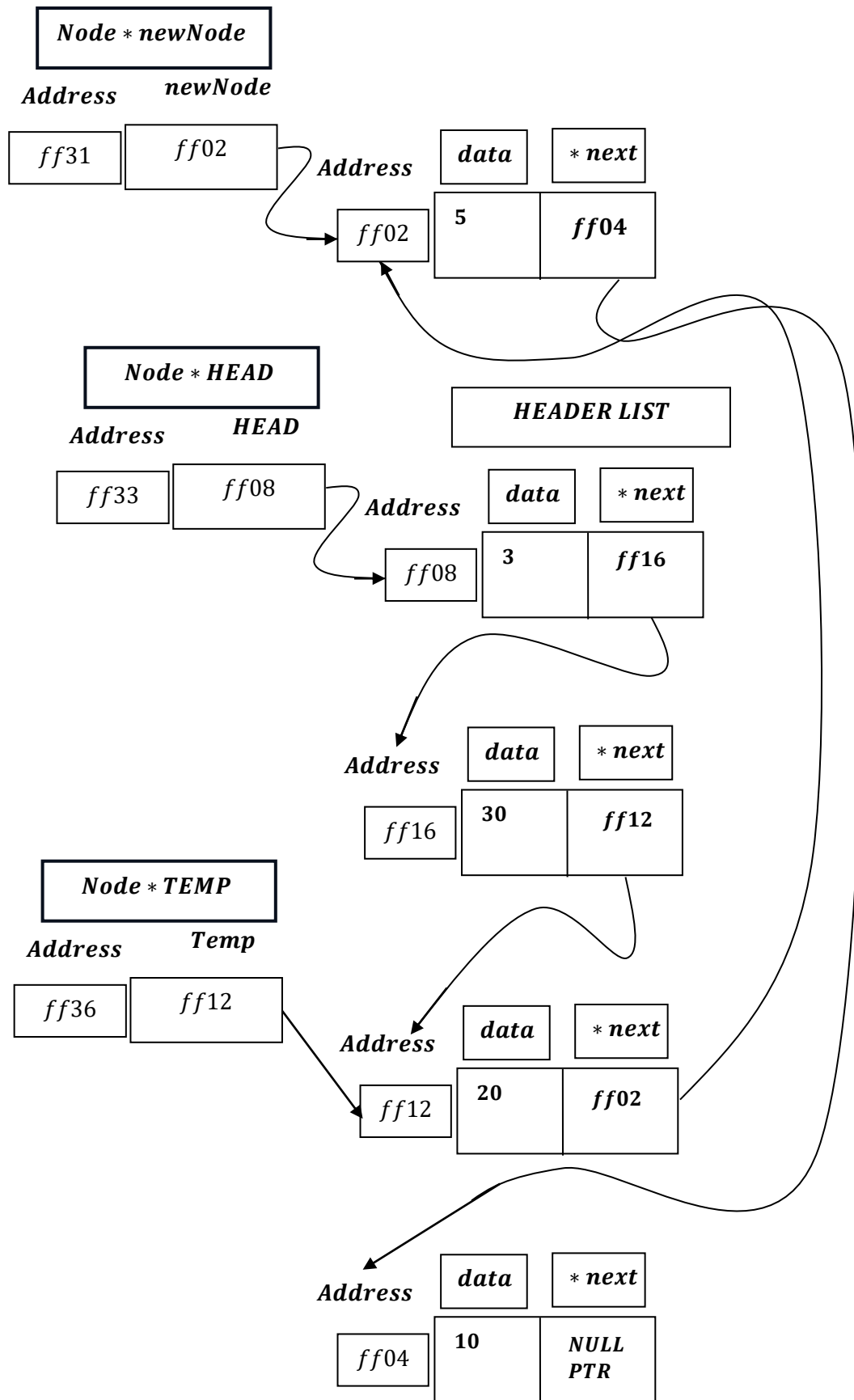
```
newNode->next = temp->next;
```

$newNode \rightarrow next = temp \rightarrow next = ff04$.

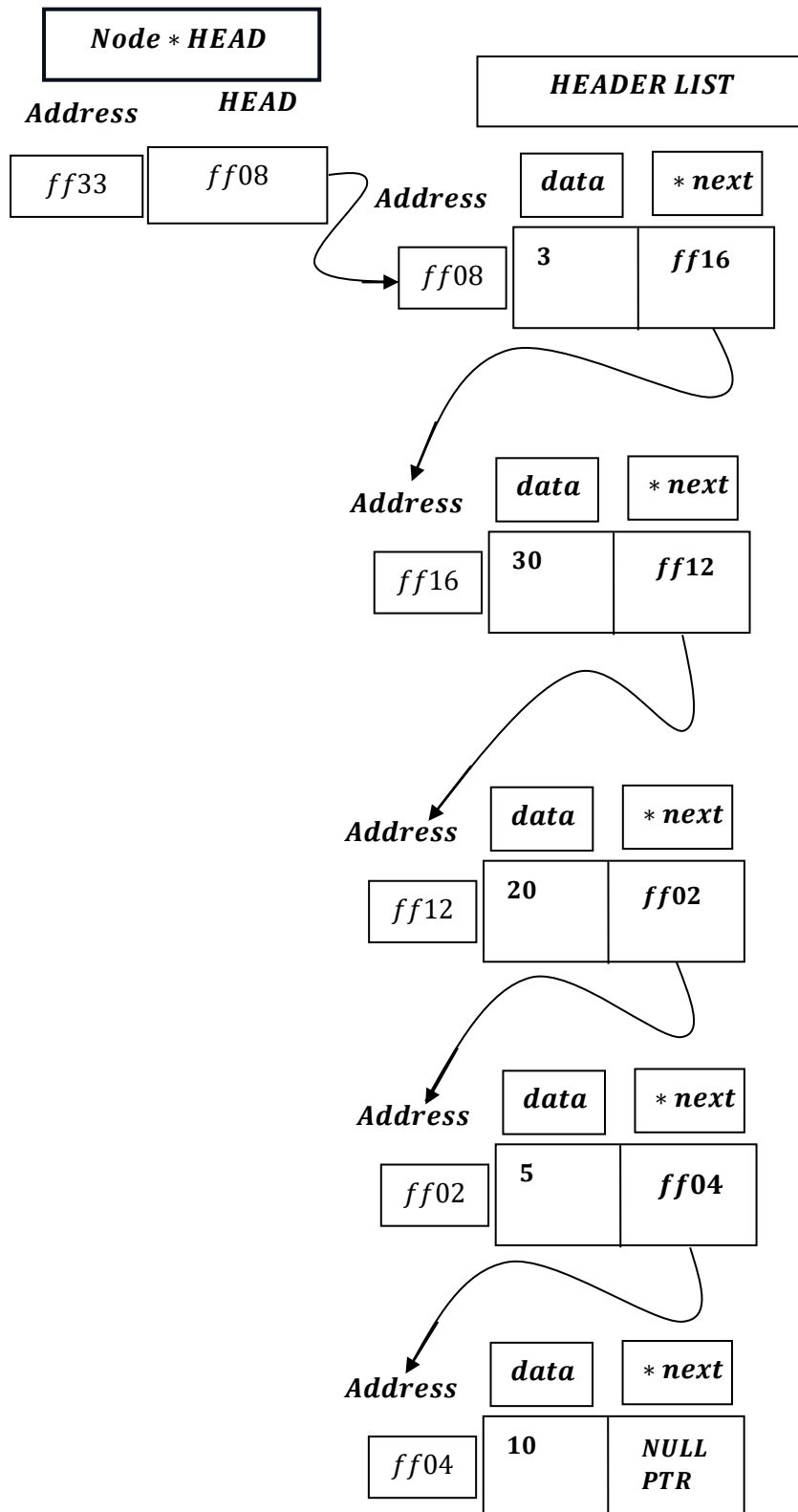


```
temp->next = newNode;
```

temp → *next* = *newNode* = *ff02*.



Hence we get our list:



`head->data++;`

$head \rightarrow data = head \rightarrow data + 1 = 3 + 1 = 4.$

